TradeLog — Characterization & Summary

Course project · Track 3 (Software Product – AI App for Investors) · Ben Rubinovitz, Idan, Shahar · BGU · 2026 Live app: https://tradelog-peach.vercel.app · **Repository:** `github.com/Benru1503/tradelog` · **Research notebook:** `ml/tradelog_prediction.ipynb`

---

1. The Financial Problem

Retail investors trading across stocks, crypto, and forex face three compounding problems that TradeLog is built to address, two of them directly with AI:

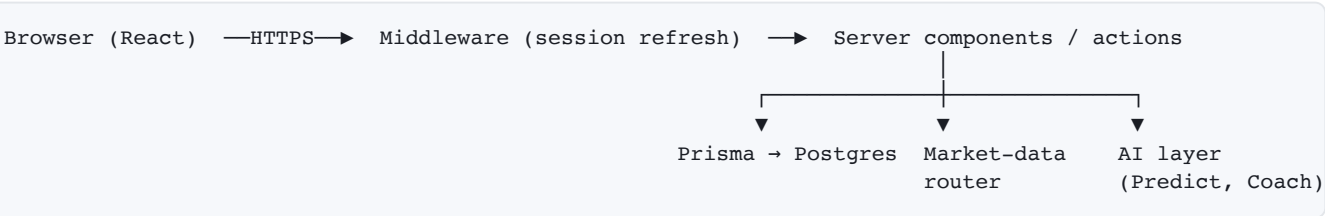
1. **No unified, honest performance view.** Positions are split across brokers and exchanges; naïve "account is up 10%" figures conflate trading skill with deposits and withdrawals, so investors cannot tell if they are actually improving.
2. **No forward-looking signal.** A self-directed investor has no accessible tool that estimates the probability a position moves up or down over the next day/week, stated with an honest confidence level rather than a black-box "buy/sell" call.
3. **No feedback on trading *behaviour*.** Outcomes are dominated by habits — holding losers too long, oversizing after a loss, ignoring one's own edge — but nothing surfaces these patterns from a trader's own history.

TradeLog's two AI features map directly onto problems 2 and 3: **Predict** (a trained direction-forecasting model with a stated confidence level) and **Coach** (an LLM that turns a trader's own history into behavioural findings). Problem 1 is solved by the underlying product itself — journaling, positions, and time-/money-weighted returns — which both AI features build on top of.

---

2. Product & Technological Architecture

TradeLog is a single Next.js 14 (App Router, TypeScript) application backed by Supabase (Postgres + Google OAuth, EU region) and deployed on Vercel in the same Frankfurt region as its database — no separate backend service. Pages are React Server Components; every mutation runs through a server action or API route, so the browser never talks to the database or holds a provider API key.



Core product surfaces: trade log with positions and averaged cost, cash-flow ledger powering TWR/MWR returns, analytics (equity curve, sector heatmap, allocation), a watchlist, and a sandboxed "Playground" for what-if/DCA simulation. Market data (Finnhub for stock/forex quotes, Yahoo Finance for historical candles, CoinGecko for crypto) is fetched server-side only and degrades to a `—` placeholder rather than crashing when a provider is unavailable.

Engineering & security. Every query is user-scoped, row-level security is enabled on every table, and the public/anon database role holds no privileges at all — a deliberate lockdown after an audit of

Supabase's default grants found tables reachable through the public API key. Provider and model keys live only in server processes, verified by scanning the deployed bundle. Sign-in is restricted to Google in application code rather than by a dashboard setting, because the auth API is reachable directly with the public key and a setting can be changed without anyone noticing. The market-data and AI paths are rate-limited per user. CI gates every push on lint, type-check, formatting, a production build, and **148 unit tests** — including golden-vector tests that pin the Python↔TypeScript model parity described in §3.1; **11 Playwright end-to-end specs** run against a real database outside CI.

The two AI models

	Predict	**Coach**
Task	Next-day / next-week price direction, with confidence	Behavioural review of a trader's own history
Technology	XGBoost (gradient-boosted trees), trained offline	Gemini (LLM), called at request time
Input	19 scale-free features from daily closes + volume	Precomputed statistics + recent journal-note excerpts
Output	P(up), direction, confidence	Structured findings (JSON, schema-validated)
Serves	Any ticker; price history fetched without an API key	The signed-in user's own trade history

---

3. The AI Models — Technical Detail

3.1 Predict — direction forecasting with confidence

Trained on 14 liquid assets (BTC, ETH, 10 large-cap stocks, SPY, QQQ), daily closes 2020→present, split strictly chronologically: train up to 2024-12, validation 2025-01→2025-09, and a held-out **test window** from 2025-10 that the model never saw during training, early stopping, or threshold selection. Two XGBoost classifiers — **d1** (next trading bar) and **w1** (next 5 bars) — are trained on 19 scale-free features (log returns, SMA ratios, Cutler's RSI, rolling volatility, distance from the 20-bar range, a volume z-score, calendar encoding, and an asset-class flag). No alternative data (on-chain, macro, sentiment) ships in the deployed model — it must score any ticker from keyless price history at request time.

Training happens in Python; inference happens in TypeScript, inside the Next.js server. The two must agree exactly, so every feature is deliberately implemented with plain loops and a finite lookback (≤50 bars) rather than library indicators or recursive smoothing (true EMA, Wilder's RSI) whose value depends on how much history happens to be fetched. The trainer exports the model as a plain-JSON tree dump plus a **measured** intercept (XGBoost's internal `base_score` representation is not trusted directly); a TypeScript evaluator walks the same trees with `float32` split comparisons, matching XGBoost's own predictor bit-for-bit. Golden-vector tests pin TypeScript features to 9 decimals and probabilities to 8 decimals against the real trained model, so a formula changed on only one side fails the suite immediately.

Confidence, precisely defined: the model outputs `p(up)`, the class probability for "closes higher after the horizon." Direction is `UP` if `p(up) ≥ 0.5` else `DOWN`; the displayed confidence is `max(p(up), 1−p(up))`, i.e. how far the model's own probability sits from a coin flip. This is the model's raw, uncalibrated probability — a genuine, honest confidence signal, but not verified against a calibration study (see §5).

Each prediction is persisted per user and later resolved against a live price into a HIT/MISS outcome, so the app's own historical track record is visible in the UI rather than only backtested numbers.

3.2 Coach — behavioural analysis via LLM

Coach answers a different question: not "what will the market do," but "what does this trader's own history say about how they trade." Every number the model can cite — win rate, hold-time asymmetry between winners and losers, payoff ratio, revenge-trade rate, position-sizing discipline, per-tag and per-weekday performance — is computed deterministically in TypeScript from the user's own Prisma rows *before* the model ever sees the data. The LLM (Gemini, `gemini-flash-latest`) receives this finished fact sheet and is instructed to interpret it, never to compute new numbers; output is constrained to a fixed JSON schema (Gemini's `responseSchema`) and independently re-validated with Zod on the way back, so a malformed or hallucinated response fails loudly rather than rendering next to real P&L. The exact fact sheet behind every report is stored and shown in the UI, so any claim in a report is auditable against the numbers that produced it.

This design directly targets the standard failure mode of LLM-generated financial content — a model asked to "analyse these trades" will confidently invent an average holding time; a model handed `avgHoldHoursLosers: 12.4` can only repeat it. Operationally, a report requires at least 5 closed trades, is cached by a hash of the fact sheet (unchanged history reuses the stored report instead of issuing another request), and each user is capped at 10 generations per rolling 24 hours, since one shared API key serves the whole install.

---

4. Results

4.1 Predict — production model (test window 2025-10 → 2026-07, 2,900–2,956 held-out rows)

Horizon	Test AUC	Test accuracy	Base rate (always "up")
d1 (next day)	0.531	51.7%	51.8%
w1 (next week)	0.527	51.7%	51.6%

Accuracy sits at or slightly below the naïve baseline; AUC modestly above 0.5 indicates a faint *ranking* signal rather than reliable directional accuracy — the honest state of daily-horizon forecasting, and stated as such in the product UI itself.

Backtest (long when `p(up) ≥ 0.55`, flat otherwise, 10 bps fee per position change), production model:

Asset	Strategy	Buy & hold	Days in market	Max drawdown	Sharpe
BTC	**+9.7%**	−46.1%	29%	−14.9%	0.54
AAPL	−5.8%	+30.5%	27%	−15.3%	−0.40
SPY	−0.1%	+12.3%	54%	−6.6%	0.04

The BTC result looks strong, and the drawdown column shows why: the mechanism is drawdown-avoidance — the model sat out most of a fall that took buy-and-hold down 46% — not directional skill. On the two assets that rallied, stepping out cost real money. This is a regime-dependent effect, not a general edge.

The research notebook (`ml/tradelog_prediction.ipynb`) independently compares three models on the identical leak-free split — XGB-lite (deployed features only), XGB-full (+ on-chain and macro data), and an LSTM:

Model	Test AUC	Accuracy	Base rate
XGB-lite	0.512	51.8%	51.8%
XGB-full	0.494	51.6%	51.8%
LSTM	0.500	51.3%	51.1%

All three sit at the edge of noise; alternative data did not help. **How fragile the backtest is, measured two ways.** The notebook applies the same backtest rules to its own alternative-data model (XGB-full) over the same price window, and BTC comes out at **−7.0%** instead of +9.7% — while buy-and-hold is identical in both runs (−46.1%, to the basis point), which proves the price data is the same and the entire gap is the choice of model. The notebook's fee sensitivity pushes the same direction: that run is −1.7% at 0 bps, −7.0% at 10 bps, −14.3% at 25 bps. So the honest answer to the course's alpha question is the notebook's: **no consistent positive alpha net of fees.** The +9.7% is one draw from a distribution over model variants and cost assumptions; the one defensible effect is drawdown avoidance in a falling market, and both the notebook and the product documentation say so explicitly.

4.2 Coach — result is a design guarantee, not an accuracy number

Coach is not a predictive model, so it has no accuracy metric to report. Its "result" is the guarantee enforced by its architecture: every quoted figure in a generated report is traceable to a real, precomputed number, verified by schema validation on every response. What is testable is that layer, and it is: the fact computation is fully unit-tested (break-even handling, soft-deleted trades, null-vs-zero semantics, payload caps), alongside a test asserting that the Gemini response schema and the Zod validator describe the same shape. In practice this reliably surfaces genuine, specific patterns — e.g. a hold-time ratio showing losers held several times longer than winners, or a cluster of revenge trades with a below-baseline win rate — rather than generic advice.

---

5. Limitations & Risks (Caveats)

Predict:

- **Single test window** (9 months, one market regime). Walk-forward evaluation across multiple windows is the right next step before trusting the edge further.
- **Threshold (0.55) chosen a priori**, not optimised, and probabilities are **uncalibrated** — the confidence number is honest but unverified against a reliability curve.
- **Model-choice sensitivity**: as shown in §4.1, swapping the deployed feature set for the notebook's alternative-data variant flips the sign of the BTC backtest on identical prices. Any single backtest number should be read with that error bar in mind.
- **Serving-time domain shift**: crypto is trained on Yahoo `BTC-USD` but served from CoinGecko at inference time; venue closes differ by tens of basis points (features are scale-free, which limits but does not eliminate the effect).
- **Live scoring is calendar-based, the label is bar-based**: a prediction resolves 24h (D1) or 7d (W1) after it was made, an approximation of the bar horizon the model was trained on; an exactly-flat outcome counts as a MISS.
- **Basket bias**: all 14 assets are large, liquid names that survived to 2026 — a mild survivorship tilt.
- Depends on Yahoo Finance's unofficial (but stable in practice) keyless endpoint; on failure the UI degrades to a standard "historical data unavailable" message rather than crashing.

Coach:

- Runs on Google AI Studio's **free tier** — request quotas are per-account and shared by all users of an install (hence the 10-reports-per-day cap), and Google's free-tier terms permit using submitted content to improve their products (the paid tier does not). Journal notes and trade metrics are sent per report; the privacy page states this and the opt-out is simply not to use the feature.
- The live network call is exercised manually, not covered by the automated test suite (the deterministic fact-computation layer is fully unit-tested).
- Free-text journal notes flow into the prompt; the system instruction constrains the model to treat them strictly as data, and output is schema-constrained with no tool use — but prompt injection is a design consideration, not a mathematically eliminated risk.

---

6. Future Work

- **Walk-forward retraining** on a schedule, replacing the single frozen 2026-07 snapshot.
- **Probability calibration study** (reliability curves) so the displayed confidence is verified, not just honestly computed.
- **Keyless macro features** (SPX/VIX/DXY) at serving time — cheap to add without breaking the "no API key" constraint.
- **Position-aware alerts** — notify a user when the model's call flips on a ticker they currently hold.
- **Additional AI surfaces under consideration**: a document-grounded assistant (retrieval-augmented Q&A over company filings) as a complementary, longer-horizon research tool alongside the existing short-horizon Predict and behavioural Coach.